

W Y K Ł A D 2

Nazwy, typy i pamięć w Pythonie

Wprowadzenie do programowania II

Informatyka Stosowana | semestr letni 2025/2026

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

- 1 Nazwy zmiennych i słowa kluczowe
- 2 Typy proste: bool, int, float, complex, str
- 3 Sprawdzanie typów i konwersje
- 4 Type hints (PEP 484)
- 5 Obiekty i pamięć: id(), referencje, GC
- 6 Mutowalność vs niemutowalność
- 7 Operatory: arytmetyczne, porównania, logiczne, bitowe
- 8 Instrukcje sterujące: if/elif/else, match/case

Nazwy w Pythonie

Identyfikatory, zmienne i słowa kluczowe

Zasady nazywania zmiennych

Dozwolone znaki

Litery (A-Z, a-z), cyfry (0-9), podkreślenie (_)

Pierwszy znak

Litera lub _ (nigdy cyfra!)

Wielkość liter

Python rozróżnia: name, Name, NAME to 3 różne zmienne

Konwencja PEP 8

snake_case dla zmiennych i funkcji

Długość

Dowolna, ale rozsądna (czytelność!)

```
user_age = 25 # OK   _private = 1 # OK   2name = 'x' # BLAD!
```

Słowa kluczowe (33 + 3 soft)

Zastrzeżone nazwy — nie można ich używać jako identyfikatorów.

```
False  None  True  and  as  assert  async  await  break  class  continue  def
del    elif  else  except  finally  for  from  global  if  import  in  is  lambda
nonlocal  not  or  pass  raise  return  try  while  with  yield
```

Soft keywords (Python 3.10+): **match** **case** **type** **_**

Można ich używać jako nazw zmiennych, ale są słowami kluczowymi w określonych kontekstach.

Typy proste

bool, int, float, complex, str

bool - typ logiczny

```
>>> True, False
(True, False)

>>> isinstance(True, int) # podtyp int!
True

>>> True + True + False
2

>>> bool(0), bool(''), bool([])
(False, False, False)

>>> bool(1), bool('x'), bool([0])
(True, True, True)
```

Falsy values:

```
False, 0, 0.0, 0j
'', ""
', "'
[], (), {}
set(), frozenset()
None, range(0)
```

Truthy:

Wszystko inne!

bool - leniwa ewaluacja (short-circuit)

```
# Operator and - zwraca pierwszy falszy LUB ostatni element
>>> 1 and 2 and 3          # wszystkie truthy -> ostatni
3
>>> 1 and 0 and 3          # 0 jest falszy -> stop
0

# Operator or - zwraca pierwszy truthy LUB ostatni element
>>> 0 or '' or 'hello'     # szuka truthy
'hello'
>>> 0 or '' or []          # wszystkie falszy -> ostatni
[]

# Praktyczne zastosowanie:
>>> name = input() or 'Anonim' # domyślna wartość
```

and i **or** nie zwracają True/False — zwracają jeden z operandów!

int - typ całkowity

```
>>> x = 1_000_000 # podkreślenia dla czytelności
>>> x
1000000

>>> 100 ** 100 # nieograniczony zakres!
1000000000000000000000000000000000000...

>>> import sys
>>> sys.getsizeof(0) # 28 bajtów!
28
>>> sys.getsizeof(10**100) # rośnie z wartością
72
```

Cechy int:

Nieograniczony
zakres (arbitrary
precision)

Obiekt niemutowalny

Podkreslenia _
jako separator
tysiecy

Minimalny rozmiar:
28 bajtów (CPython)

int - systemy liczbowe

Literały w różnych systemach:

```
>>> 0b1010          # binarny (0b)
10
>>> 0o17             # ósemkowy (0o)
15
>>> 0xFF             # szesnastkowy (0x)
255
```

Konwersja na string w danym systemie:

```
>>> bin(42)          # -> '0b101010'
>>> oct(42)          # -> '0o52'
>>> hex(42)          # -> '0x2a'

>>> int('FF', 16)    # parsing z podana baza
255
```

float - typ zmiennoprzecinkowy

```
>>> 3.14
>>> 2.0e-3          # notacja naukowa
0.002

>>> float('inf')    # nieskończoność
>>> float('-inf')
>>> float('nan')    # Not a Number

>>> import math
>>> math.isnan(float('nan'))
True
>>> math.isinf(float('inf'))
True
```

IEEE 754

double precision
(64-bit)

1 bit – znak
11 bitów – wykładnik
52 bity – mantysa

Precyzja: ~15–17
cyfr znaczących

**Uwaga: 0.1 + 0.2
nie jest dokładnie 0.3!**

Precyzyjne obliczenia: decimal i fractions

decimal

```
from decimal import Decimal

>>> Decimal('0.1') + Decimal('0.2')
Decimal('0.3')

>>> Decimal('0.1') + Decimal('0.2')
    == Decimal('0.3')
True

# Dokładna arytmetyka dziesiętna
# - brak błędów zaokrągleń
```

fractions

```
from fractions import Fraction

>>> Fraction(1, 3)
Fraction(1, 3)

>>> Fraction(1,3) + Fraction(1,6)
Fraction(1, 2)

>>> Fraction('0.1') * 3
Fraction(3, 10)
```

complex - liczby zespolone

```
>>> z = 2 + 3j          # j, nie i!  
>>> z.real             # część rzeczywista  
2.0  
>>> z.imag            # część urojona  
3.0  
>>> z.conjugate()      # sprzężenie  
(2-3j)  
>>> abs(z)             # moduł: sqrt(4+9)  
3.605551275463989
```

Typ complex przydaje się w obliczeniach naukowych, inżynierskich i DSP (przetwarzanie sygnałów).

Moduł cmath rozszerza math o wersje dla liczb zespolonych.

str - łańcuchy znakowe

Trzy sposoby tworzenia:

```
s1 = 'apostrofy'
```

```
s2 = "cudzyslowy"
```

```
s3 = """wieloliniowy  
string"""
```

Znaki specjalne:

```
>>> 'linia1\nlinia2'          # \n = nowa linia
```

```
>>> 'tab\tznak'              # \t = tabulator
```

Raw string - brak interpretacji \:

```
>>> print(r'C:\Users\name')  # r prefix
```

```
C:\Users\name
```

Operacje:

```
>>> 'Py' + 'thon'
```

```
# konkatencja: 'Python'
```

```
>>> 'Ha' * 3
```

```
# powtórzenie: 'HaHaHa'
```

str - indeksowanie i slicing

+	-	-	+	-	-	+	-	-	+	-	-	+
	P		y		t		h		o		n	
+	-	-	+	-	-	+	-	-	+	-	-	+
0	1	2	3	4	5							
-6	-5	-4	-3	-2	-1							

indeksy dodatnie
indeksy ujemne

```
>>> s = "Python"
>>> s[0]      # 'P'           >>> s[-1]    # 'n'
>>> s[0:3]    # 'Pyt'        >>> s[2:]     # 'thon'
>>> s[::-1]   # 'nohtyP'     >>> s[::-2]   # 'Pto'
```

Składnia: s[start:stop:step] – stop jest WYŁĄCZONY

str - przydatne metody

Metoda	Opis
<code>s.upper()</code> , <code>s.lower()</code>	Zmiana wielkości liter
<code>s.strip()</code>	Usuwa białe znaki z krańców
<code>s.split(sep)</code>	Dzieli na listę wg separatora
<code>sep.join(lista)</code>	Łączy listę w string z separatorem
<code>s.replace(old, new)</code>	Zamienia podciąg
<code>s.find(sub)</code> / <code>s.index(sub)</code>	Szuka pozycji podciągu (-1 / ValueError)
<code>s.startswith(p)</code> / <code>s.endswith(p)</code>	Sprawdza początek / koniec
<code>s.count(sub)</code>	Liczy wystąpienia podciągu
<code>s.isdigit()</code> / <code>s.isalpha()</code>	Sprawdza, czy cyfry / litery
<code>len(s)</code>	Długość lancucha (funkcja wbudowana)

String jest niemutowalny — każda metoda zwraca NOWY string.

str - Unicode, ord() i chr()

```
>>> ord('a')           # kod Unicode znaku
97
>>> chr(97)            # znak o danym kodzie
'a'

>>> chr(9731)          # Unicode: bałwanek!
'\u2603'
>>> chr(128013)        # emoji wąż (Python!)
'\U0001f40d'

>>> '\u0050\u0079\u0074\u0068\u006f\u006e'
'Python'
```

Python 3 używa Unicode (UTF-8) domyślnie. Każdy string to sekwencja kodów Unicode, nie bajtów. Zakres ord/chr: 0..1114111.

Sprawdzanie typów

`type()`, `isinstance()`, konwersje i type hints

type() i isinstance()

type()

```
>>> type(42)
<class 'int'>

>>> type('abc')
<class 'str'>

>>> type(42) == int
True

# NIE uwzględnia dziedziczenia!
```

isinstance()

```
>>> isinstance(42, int)
True

>>> isinstance(True, int)
True

# Krotka typów:
>>> isinstance(42, (int, float))
True

# UWZGLĘDNIA dziedziczenie!
```

W praktyce preferowane: `isinstance()` — respektuje hierarchię typów.

Konwersje typów (casting)

```
# int <-> float <-> str
>>> int(3.7)           # 3 (obcina, nie zaokrągla!)
>>> int('42')         # 42
>>> float('3.14')     # 3.14
>>> str(42)           # '42'

# bool -> int
>>> int(True)          # 1
>>> int(False)         # 0

# Uwaga na błędy:
>>> int('3.14')       # ValueError!
>>> int('abc')         # ValueError!
>>> int(float('3.14')) # 3 - dwuetapowo OK
```

Type hints - adnotacje typów (PEP 484)

```
# Adnotacje nie wymuszają typów – to wskazówki!

name: str = 'Alice'
age: int = 25
scores: list[float] = [9.5, 8.0]

def powitanie(imie: str, wiek: int) -> str:
    return f"Czesc {imie}, masz {wiek} lat"

# Python NIE zgłosi błędu przy niezgodności!
powitanie(42, 'test')          # działa, mimo złych typów
```

Do czego służą? Czytelność kodu, podpowiedzi w IDE (IntelliSense), analiza statyczna (mypy, Pylance).

Nie zmieniają zachowania programu w runtime!

Obiekty i pamięć

id(), referencje, garbage collector

id() i sys.getsizeof()

```
>>> a = 'hello'
>>> id(a)                                # unikalny adres obiektu
140234567890
>>> hex(id(a))                           # jako hex
'0x20adc5e332'

>>> import sys
>>> sys.getsizeof(0)                       # int: 28 bajtów
28
>>> sys.getsizeof('')                     # pusty str: 49 bajtów
49
>>> sys.getsizeof('abc')                   # str 3 znaki: 52 bajty
52
>>> sys.getsizeof([])                     # pusta lista: 56 bajtów
56
```

Zmienne to etykiety, nie pudełka

```
>>> a = "xyz"
>>> id(a)
140234567890

>>> b = a                # ta sama ref!
>>> id(b)
140234567890
>>> a is b                # True!

>>> a = "aaa"            # nowy obiekt!
>>> id(a)
140234999000
>>> a is b                # False!
```

Model pamięci:

```
a = 'xyz'
a -----> [obiekt 'xyz']

b = a
a -----> [obiekt 'xyz']
b -----/

a = 'aaa'
a -----> [obiekt 'aaa']
b -----> [obiekt 'xyz']
```


Niemutowalność - każda zmiana = nowy obiekt

```
>>> x = 10
>>> id(x)                # adres obiektu 10
140234000100

>>> x = x + 1            # NIE modyfikuje 10!
>>> x
11
>>> id(x)                # INNY adres!
140234000132

# Tak samo ze stringami:
>>> s = "hello"
>>> s_id = id(s)
>>> s = s + " world"    # nowy obiekt!
>>> id(s) == s_id
False
```

Cache małych intów [-5..256]

```
# CPython tworzy wstepnie obiekty int z zakresu [-5, 256]

>>> a = 256
>>> b = 256
>>> a is b                                # True - ten sam obiekt z cache
True

>>> a = 257
>>> b = 257
>>> a is b                                # False - poza cache!
False

>>> a == b                                # True - wartosci sa rowne
True
```

To jest szczegół implementacji CPythona, nie część specyfikacji języka.
Nigdy nie polegać na **is** do porównywania wartości!

Zarządzanie pamięcią: reference counting + GC

Reference counting

Każdy obiekt ma licznik referencji. Gdy spadnie do 0, obiekt jest natychmiast zwalniany.

Garbage collector

Wykrywa cykliczne referencje ($A \rightarrow B \rightarrow A$), których ref. counting nie obsługuje. Działa okresowo.

del

Usuwa nazwę (referencję), NIE obiekt bezpośrednio. Obiekt zniknie dopiero gdy refcount = 0.

Operator

Arytmetyczne, porównania, logiczne, bitowe, przypisania

Operatory arytmetyczne

Operator	Opis	Przykład
<code>+</code>	Dodawanie	$3 + 2 = 5$
<code>-</code>	Odejmowanie	$3 - 2 = 1$
<code>*</code>	Mnożenie	$3 * 2 = 6$
<code>/</code>	Dzielenie (float!)	$7 / 2 = 3.5$
<code>//</code>	Dzielenie całkowite	$7 // 2 = 3$
<code>%</code>	Reszta (modulo)	$7 \% 2 = 1$
<code>**</code>	Potęgowanie	$2 ** 10 = 1024$
<code>divmod(a,b)</code>	Iloraz i reszta	$\text{divmod}(7,2) = (3,1)$

Kolejność: `**` > `*`, `/`, `//`, `%` > `+`, `-` (nawiasy zmieniają kolejność)

Operatory porównania

<code>==</code>	Równość wartości	<code>3 == 3 -> True</code>
<code>!=</code>	Nierówność	<code>3 != 4 -> True</code>
<code><, <=</code>	Mniejszy (lub równy)	<code>2 < 3 -> True</code>
<code>>, >=</code>	Większy (lub równy)	<code>3 >= 3 -> True</code>
<code>is</code>	Tożsamość obiektu (adres)	<code>a is b</code>
<code>is not</code>	Brak tożsamości	<code>a is not None</code>
<code>in</code>	Należność do kolekcji	<code>'a' in 'abc' -> True</code>
<code>not in</code>	Brak należności	<code>5 not in [1,2,3] -> True</code>

Przypomnienie: **is** porównuje adresy (id), **==** porównuje wartości. Używaj **is** tylko z **None**.

Operatory logiczne: and, or, not

```
# not - negacja
>>> not True           # False
>>> not 0              # True  (0 jest falsy)

# and - koniunkcja (zwraca operand, nie True/False!)
>>> 'a' and 'b'        # 'b'
>>> '' and 'b'         # ''   (pusty = falsy, stop)

# or - alternatywa (zwraca operand!)
>>> '' or 'domyslne'   # 'domyslne'
>>> 'wartosc' or 'domyslne' # 'wartosc'

# Priorytet: not > and > or
```

Operatory bitowe

&	AND bitowy	<code>0b1010 & 0b1100 = 0b1000 (8)</code>
 	OR bitowy	<code>0b1010 0b1100 = 0b1110 (14)</code>
^	XOR bitowy	<code>0b1010 ^ 0b1100 = 0b0110 (6)</code>
~	NOT bitowy	<code>~0b1010 = -11 (inwersja + 1)</code>
<<	Przesunięcie w lewo	<code>1 << 3 = 8 (= 1 * 2**3)</code>
>>	Przesunięcie w prawo	<code>16 >> 2 = 4 (= 16 // 4)</code>

Operatory bitowe działają na binarnej reprezentacji intów. Przydatne w kryptografii, sieci, embedded.

Operatory przypisania i walrus operator

Przypisanie rozszerzone

```
x += 1      # x = x + 1
x -= 2      # x = x - 2
x *= 3      # x = x * 3
x /= 4      # x = x / 4
x //= 2     # x = x // 2
x **= 2     # x = x ** 2
x %= 3      # x = x % 3
```

Walrus := (PEP 572, 3.8+)

Przypisanie w wyrażeniu:

Bez walrus:

```
n = len(data)
if n > 10: print(n)
```

Z walrus:

```
if (n := len(data)) > 10:
    print(n)
```

Walrus operator oszczędza jedną linię i jedno wywołanie — przydatny w while, if, list comprehension.

Instrukcje sterujące

if / elif / else, match / case

if / elif / else

Podstawowa składnia

```
x = int(input("Liczba: "))

if x < 0:
    print('ujemna')
elif x == 0:
    print('zero')
elif x < 100:
    print('mala dodatnia')
else:
    print('duza')
```

Operator trójskładnikowy

```
# wartosc_true if warunek else
wartosc_false
```

```
>>> x = 10
>>> wynik = "duzo" if x > 5
                else "malo"
>>> wynik
'duzo'
```

```
# Można zagnieżdżać:
>>> "A" if x>90 else "B"
        if x>70 else "C"
```

match / case (Python 3.10+)

```
# Structural Pattern Matching (PEP 634)
```

```
def http_status(code):  
    match code:  
        case 200:  
            return "OK"  
        case 301 | 302:  
            return "Redirect"  
        case 404:  
            return "Not Found"  
        case 500:  
            return "Server Error"  
        case _:  
            return "Unknown"
```

```
# _ to wildcard - dopasowuje wszystko (jak else)
```

Pułapka: truthy/falsy w warunkach

```
# Structural Pattern Matching (PEP 634)
```

```
def http_status(code):  
    match code:  
        case 200:  
            return "OK"  
        case 301 | 302:  
            return "Redirect"  
        case 404:  
            return "Not Found"  
        case 500:  
            return "Server Error"  
        case _:  
            return "Unknown"
```

```
# _ to wildcard - dopasowuje wszystko (jak else)
```

PEP 8: porównuj z `None` za pomocą `is` / `is not`, nigdy `==`. Dla pustych kolekcji używaj `not x`.

Pułapka: ukryte konwersje typów

```
# Często błąd – sprawdzanie pustej listy:
data = []

if len(data) == 0:           # działa, ale nie-pythonowe
if data == []:              # tworzy nową pustą listę!
if not data:                # Pythonowy sposób!

# Sprawdzanie None:
if x == None:               # działa, ale ŹLE
if x is None:               # jedyny poprawny sposób!

# Uwaga: 0, '', [] sa falsy - ale to NIE jest None!
```

Sztuczki z typami

```
# Zwielokrotnianie stringow jako ASCII art:
>>> print('#' * 40)          # szybki separator

# F-string z formatowaniem:
>>> f'{42:08b}'              # '00101010' (binarnie)
>>> f'{255:#06x}'            # '0x00ff' (hex)
>>> f'{1000000:_}'           # '1_000_000'

# sum() z boolami:
>>> data = [3, -1, 4, -2, 5]
>>> sum(x > 0 for x in data) # 3 (ile dodatnich?)

# Infinity jako wartosc startowa:
>>> minimum = float('inf')
>>> float('inf') > 10**308    # True (wieksze od wszystkiego)
```

Podsumowanie

- 1 Nazwy: snake_case, 33 słów kluczowych, wielkość liter ma znaczenie
- 2 Typy proste: bool (podtyp int), int (nieograniczony), float (IEEE 754), complex, str
- 3 type() vs isinstance() — preferuj isinstance() (respektuje dziedziczenie)
- 4 Type hints (PEP 484) — nie wymuszają, ale pomagają IDE i linterom
- 5 Zmienne to referencje, nie pudełka — id() pokazuje adres obiektu
- 6 Niemutowalne: int, float, str, tuple — każda zmiana = nowy obiekt
- 7 Operatory: arytmetyczne, porównania, logiczne, bitowe, walrus :=
- 8 and/or zwracają operand, nie True/False (short-circuit evaluation)